

Handwritten Digit Recognition using CNN (MNIST) with PyTorch

Mostafa Badawe
ID: 2023019029
AI3

May 17, 2026

Contents

1	Abstract	3
2	Introduction	3
3	Dataset Description	3
3.1	Dataset Information	3
4	Data Preprocessing	3
4.1	Preprocessing Steps	4
4.2	Normalization Formula	4
5	CNN Model Architecture	4
5.1	Architecture Layers	4
6	Mathematical Equations	4
6.1	Convolution Operation	4
6.2	ReLU Activation Function	5
6.3	MaxPooling Operation	5
6.4	Softmax Function	5
6.5	Cross Entropy Loss	5
7	Training Process	5
8	Experiments	6
8.1	Experiment 1	6
8.2	Experiment 2	6
9	Results	6
9.1	Experiment Comparison Table	6
9.2	Optimizer Comparison	6
10	Classification Report	7

11 Confusion Matrix	7
11.1 Accuracy Formula	7
12 Discussion	7
13 Conclusion	7
14 Future Improvements	8
15 Technologies Used	8
16 References	8

1 Abstract

This project presents a Convolutional Neural Network (CNN) model implemented using PyTorch for handwritten digit recognition on the MNIST dataset.

The project aims to classify handwritten digits from 0 to 9 with high accuracy using deep learning techniques. Two optimization algorithms, Adam and SGD, were compared to evaluate their effect on model performance.

The experimental results showed that the Adam optimizer achieved superior performance with a testing accuracy of 99.23%.

2 Introduction

Handwritten digit recognition is one of the most important applications of computer vision and deep learning. It is widely used in:

- Banking systems
- Postal code recognition
- Optical Character Recognition (OCR)
- Document processing

Convolutional Neural Networks (CNNs) are highly effective for image classification tasks because they automatically extract spatial features from images.

In this project, a CNN model was built using PyTorch to classify handwritten digits from the MNIST dataset.

3 Dataset Description

The project uses the MNIST dataset.

3.1 Dataset Information

Feature	Value
Dataset Name	MNIST
Training Images	60,000
Testing Images	10,000
Image Size	28×28
Color Type	Grayscale
Classes	10 Digits (0–9)

Table 1: MNIST Dataset Information

4 Data Preprocessing

Several preprocessing techniques were applied before training the CNN model.

4.1 Preprocessing Steps

1. Convert images to tensors
2. Normalize pixel values
3. Split dataset into training and validation sets
4. Create DataLoaders

4.2 Normalization Formula

The normalization process is represented as:

$$X_{normalized} = \frac{X - \mu}{\sigma}$$

Where:

- X = Original pixel value
- μ = Mean
- σ = Standard deviation

5 CNN Model Architecture

The CNN model consists of multiple layers designed for feature extraction and classification.

5.1 Architecture Layers

Layer	Description
Conv2D	$1 \rightarrow 32$ Filters
ReLU	Activation Function
MaxPooling	Downsampling
Conv2D	$32 \rightarrow 64$ Filters
ReLU	Activation Function
MaxPooling	Downsampling
Flatten	Feature Vector Conversion
Dense Layer	$3136 \rightarrow 128$
Dropout	0.5
Output Layer	10 Classes

Table 2: CNN Architecture

6 Mathematical Equations

6.1 Convolution Operation

The convolution operation is defined as:

$$Y(i, j) = \sum_m \sum_n X(i - m, j - n) K(m, n)$$

Where:

- X = Input image
- K = Kernel/filter
- Y = Output feature map

6.2 ReLU Activation Function

The ReLU activation function is:

$$f(x) = \max(0, x)$$

6.3 MaxPooling Operation

MaxPooling selects the maximum value:

$$Y = \max(x_1, x_2, \dots, x_n)$$

6.4 Softmax Function

The Softmax function converts outputs into probabilities:

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

6.5 Cross Entropy Loss

The loss function used during training:

$$L = - \sum_{i=1}^n y_i \log(\hat{y}_i)$$

Where:

- y_i = True label
- $\hat{y}_i$ = Predicted probability

7 Training Process

The model was trained using:

- Loss Function: CrossEntropyLoss
- Epochs: 10

- Framework: PyTorch

The following metrics were monitored:

- Training Loss
- Validation Loss
- Training Accuracy
- Validation Accuracy

8 Experiments

Two experiments were performed using different optimizers and batch sizes.

8.1 Experiment 1

- Optimizer: Adam
- Batch Size: 64

8.2 Experiment 2

- Optimizer: SGD
- Batch Size: 128

9 Results

9.1 Experiment Comparison Table

Model	Optimizer	Batch Size	Accuracy	Loss
Model A	Adam	64	99.23%	0.0283
Model B	SGD	128	97.44%	0.0804

Table 3: Experiment Results Comparison

9.2 Optimizer Comparison

Feature	Adam	SGD
Convergence Speed	Fast	Slower
Accuracy	Higher	Lower
Training Stability	Better	Moderate
Learning Rate	Adaptive	Fixed

Table 4: Adam vs SGD Comparison

10 Classification Report

The classification report showed excellent performance.

Metric	Value
Precision	0.99
Recall	0.99
F1-Score	0.99
Accuracy	99.23%

Table 5: Classification Metrics

11 Confusion Matrix

The confusion matrix demonstrated that most handwritten digits were correctly classified with very low error rates.

11.1 Accuracy Formula

$$Accuracy = \frac{Correct\ Predictions}{Total\ Predictions} \times 100$$

12 Discussion

The Adam optimizer achieved the best results because it adapts learning rates automatically during training.

Advantages of Adam:

- Faster convergence
- Higher accuracy
- Better optimization performance

Advantages of SGD:

- Simpler optimization method
- Lower memory usage

However, SGD required more epochs and tuning to achieve good performance.

13 Conclusion

This project successfully implemented a CNN model using PyTorch for handwritten digit recognition.

The model achieved excellent results on the MNIST dataset with a testing accuracy of 99.23%.

The experiments showed that the Adam optimizer performed better than SGD in terms of convergence speed and final accuracy.

14 Future Improvements

Possible future improvements include:

- Using Batch Normalization
- Applying Data Augmentation
- Using deeper CNN architectures
- Hyperparameter tuning
- Deploying the model as a web application

15 Technologies Used

- Python
- PyTorch
- Torchvision
- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn
- Google Colab

16 References

1. PyTorch Documentation: <https://pytorch.org/>
2. Torchvision Documentation: <https://pytorch.org/vision/stable/>
3. MNIST Dataset: <http://yann.lecun.com/exdb/mnist/>
4. Deep Learning with Python — Francois Chollet